



# Applied ML

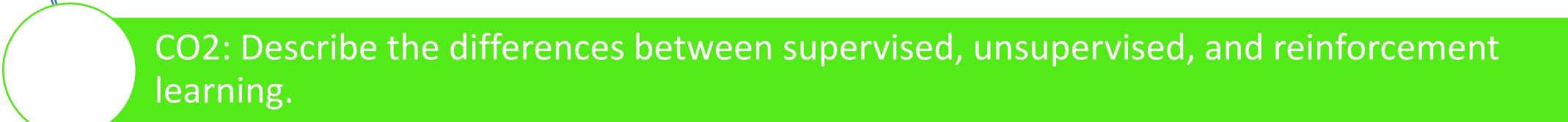
|                  |                |
|------------------|----------------|
| Prerequisite(s): | Elements of AI |
|------------------|----------------|

|                              |                         |
|------------------------------|-------------------------|
| Total Units to be Covered: 6 | Total Contact Hours: 48 |
|------------------------------|-------------------------|

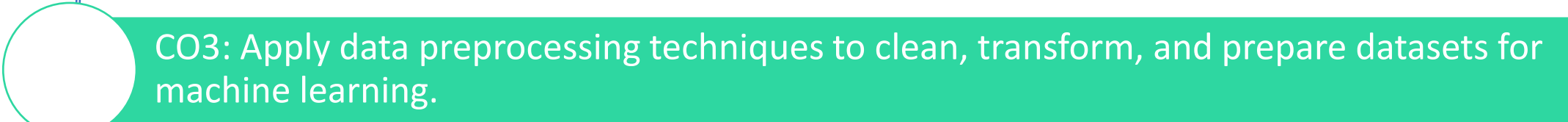
# Course Outcomes



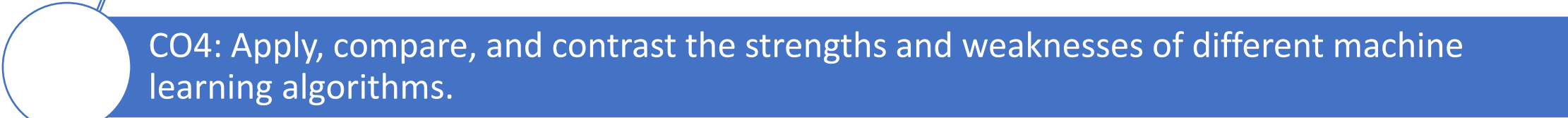
CO1: Recall and define key machine learning concepts, terminologies, and algorithms.



CO2: Describe the differences between supervised, unsupervised, and reinforcement learning.



CO3: Apply data preprocessing techniques to clean, transform, and prepare datasets for machine learning.



CO4: Apply, compare, and contrast the strengths and weaknesses of different machine learning algorithms.

## **Unit III: Optimizer function**

Stochastic gradient descent, Mini-Batch Gradient Descent, Momentum, Adaptive gradient algorithm (Adagrad), Adam (Adaptive Moment Estimation), RMSprop (Root Mean Square Propagation), Adadelata

# Why Do We Need Gradient Descent?

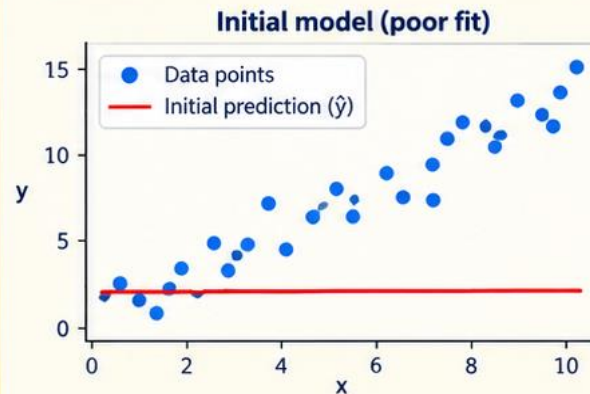
An iterative method to find the best model parameters by minimising prediction error.

## The Problem

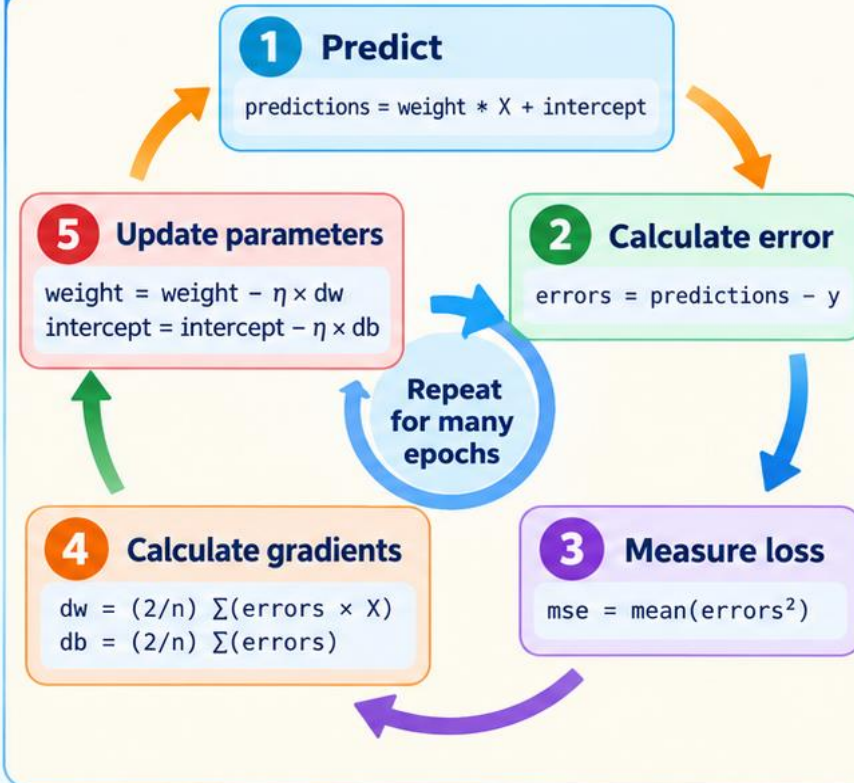
Model:  $\hat{y} = wx + b$

At the beginning:  
weight = 0.0  
intercept = 0.0

Predictions are inaccurate  
and MSE is high.

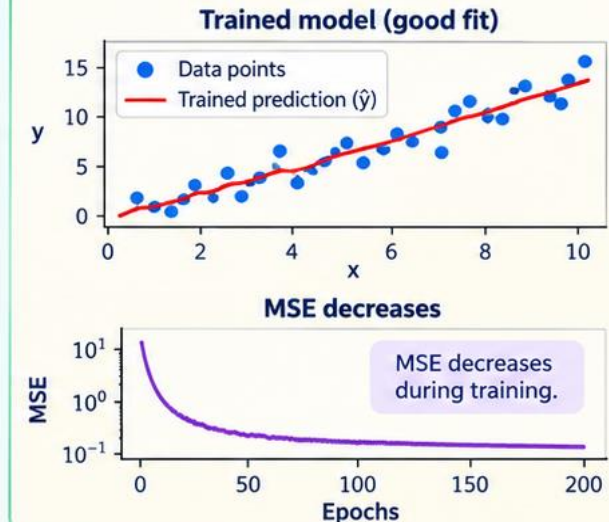


## Gradient Descent Learning Loop



## The Result

- ✓ Lower prediction error
- ✓ Better weight and intercept
- ✓ Regression line fits the data
- ✓ Model can predict new values



**Gradient descent automatically finds parameter values that minimise prediction error:**

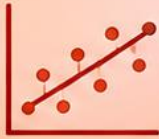
Useful for large datasets • many features • neural networks • models without a simple direct solution



# Linear Regression

Model a continuous target as a linear combination of features.

## Without regularisation



May overfit when features are many or highly correlated.

## With regularisation



Add a penalty to the loss function to constrain model coefficients and reduce overfitting.

### Ridge Regression — L2



L2

Shrinks coefficients; usually keeps all features.

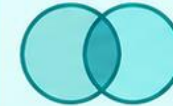
### Lasso Regression — L1



L1

Can reduce some coefficients to zero; performs feature selection.

### Elastic Net — L1 + L2



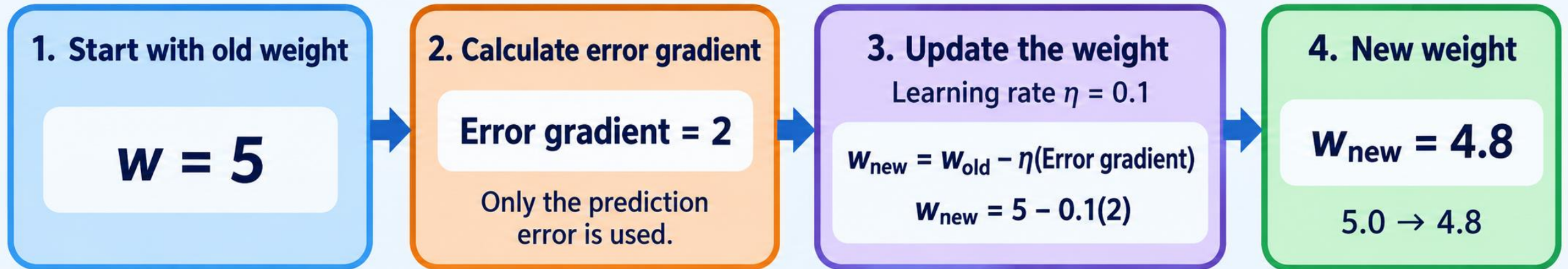
L1 + L2

Combines sparsity of Lasso with stability of Ridge.



Regularisation controls model complexity and helps improve generalisation.

# Gradient Descent Without Regularisation



## Gradient Descent Without Regularisation: Moving Down the Loss Curve





**Takeaway:** Gradient descent repeatedly moves the weight in the direction that reduces prediction error.



**Gradient descent updates model parameters using only the training set.**



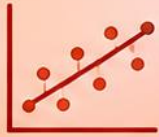
**Validation helps make decisions. Test data checks the final model.**



# Linear Regression

Model a continuous target as a linear combination of features.

## Without regularisation



May overfit when features are many or highly correlated.

## With regularisation



Add a penalty to the loss function to constrain model coefficients and reduce overfitting.

### Ridge Regression — L2



L2

Shrinks coefficients; usually keeps all features.

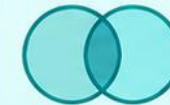
### Lasso Regression — L1



L1

Can reduce some coefficients to zero; performs feature selection.

### Elastic Net — L1 + L2



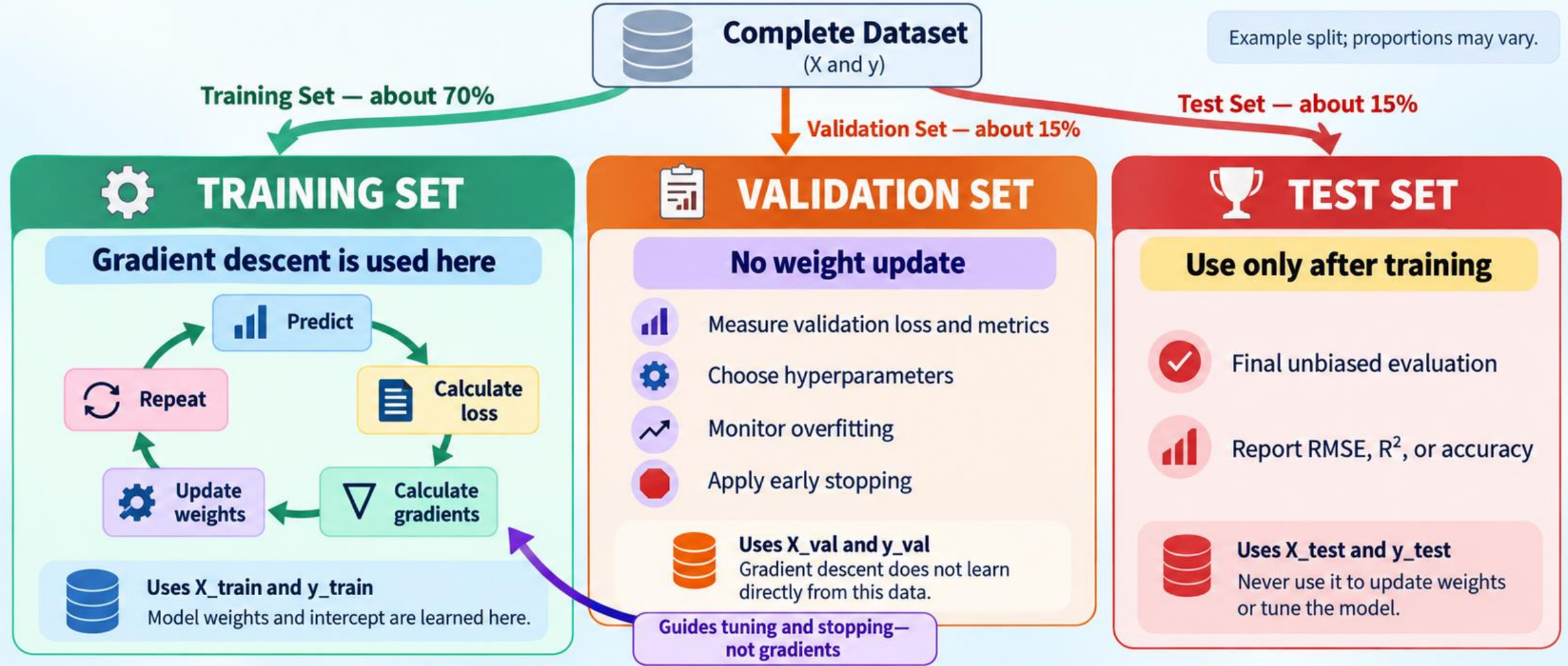
L1 + L2

Combines sparsity of Lasso with stability of Ridge.



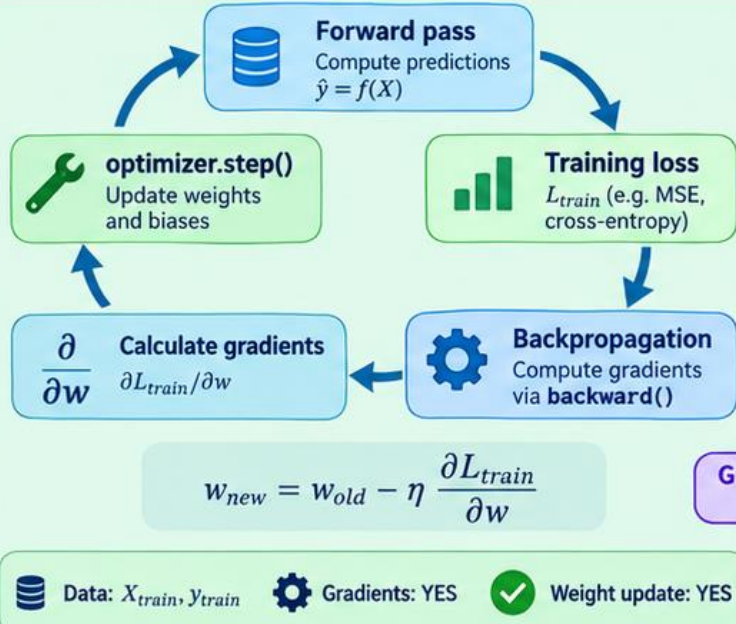
Regularisation controls model complexity and helps improve generalisation.

# Where Is Gradient Descent Used in Data Partitioning?



## 1. TRAINING — Optimizer Works Here

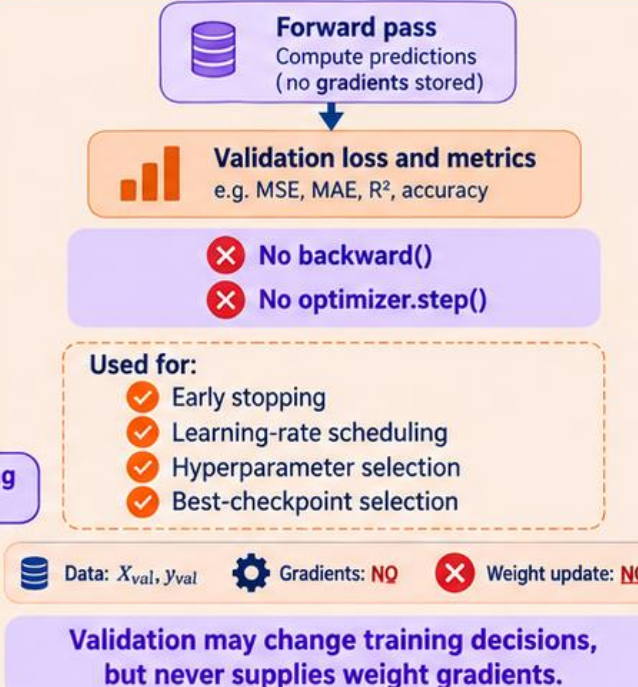
The optimizer uses gradients from the training loss to update model parameters.



Guides training indirectly

## 2. VALIDATION — Check and Guide

Monitor performance on unseen validation data to guide training decisions.



## 3. TESTING — Final Evaluation

Evaluate the selected final model on completely unseen test data.



| Aspect                    | Training         | Validation    | Test         |
|---------------------------|------------------|---------------|--------------|
| Forward pass              | YES              | YES           | YES          |
| Calculate gradients       | YES              | NO            | NO           |
| Optimizer updates weights | YES              | NO            | NO           |
| Model decisions           | Learn parameters | Tune and stop | Final report |

**Warning!** Using validation or test data to update weights causes data leakage.



**Training data teaches the model. Validation data guides the process. Test data evaluates the final model.**



# Hyperparameters of Gradient Descent

Small choices.  
Big learning.



## Learning Rate ( $\eta$ )

Controls the size of each weight update.

Too small: slow |  
Too large: may overshoot

Examples: 0.1, 0.01, 0.001



## Epochs

Maximum number of complete passes through the data.

Too few: underfitting |  
Too many: wasted training

Examples: 100, 500, 1000



## Batch Size

Number of samples used for one weight update.

1 = SGD | 32–256 = Mini-batch |  
All = Batch GD



## Learning-Rate Schedule

Changes the learning rate during training.

Types: constant, inverse scaling, adaptive



## Tolerance

Minimum improvement required to continue.

Smaller value allows more precise convergence.

Examples:  $10^{-3}$ ,  $10^{-4}$



## Patience

Number of epochs allowed without improvement.

Used for early stopping.

Examples: 5, 10, 20



## Regularisation Strength ( $\alpha$ )

Controls the penalty on large weights.

Too small: weak control |  
Too large: underfitting

Examples: 0.0001, 0.001, 0.01



## Momentum

Uses previous updates to accelerate learning.

Helps reduce oscillation.

Examples: 0.9, 0.99



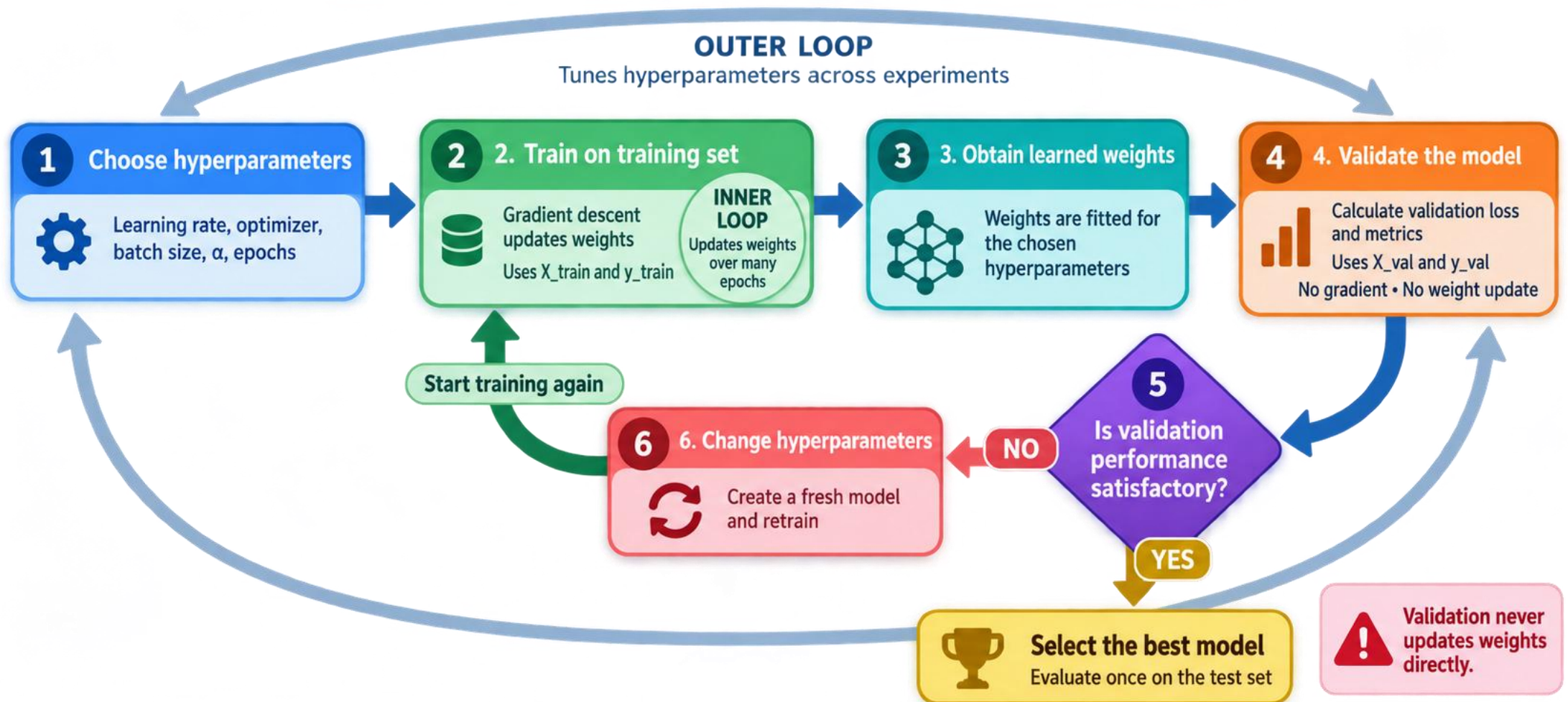
## Weight update:

$$w_{new} = w_{old} - \eta \times \text{gradient}$$



The most influential choices are learning rate, epochs, batch size, and regularisation strength.

# Training-Validation Hyperparameter Loop



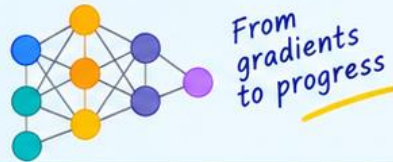
Training learns weights. Validation evaluates choices. Poor validation results lead to new hyperparameters and retraining.

# OPTIMIZERS USED FOR TRAINING

How neural networks update weights to reduce loss



Better models brighter futures



## 1 STOCHASTIC GRADIENT DESCENT (SGD)

**How it works:** Updates weights using ONE randomly selected training example.

**Update:**  $w \leftarrow w - \eta \nabla L_i(w)$

**Advantages:** Fast updates • Low memory • Can escape shallow minima

**Limitations:** Noisy path • Loss fluctuates • May converge slowly

**Best for:** Very large or streaming datasets



## 2 MINI-BATCH GRADIENT DESCENT

**How it works:** Updates weights using a small batch, commonly 32–256 examples.

**Update:**  $w \leftarrow w - \eta (\text{average batch gradient})$

**Advantages:** Stable and efficient • GPU-friendly • Most widely used

**Limitations:** Batch size must be selected • Still has some noise

**Best for:** Most deep-learning training



## 3 MOMENTUM

**How it works:** Adds a fraction of the previous update to build speed.

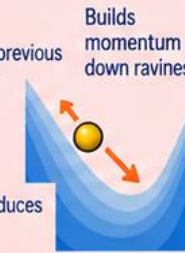
$$v_t \leftarrow \beta v_{t-1} + \nabla L(w_t)$$

$$w_{t+1} \leftarrow w_t - \eta v_t$$

**Advantages:** Faster convergence • Reduces zig-zagging

**Limitations:** Extra  $\beta$  hyperparameter • May overshoot

**Best for:** Ravines and consistent gradient directions



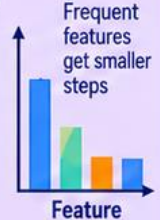
## 4 ADAGRAD

**How it works:** Gives each parameter its own learning rate; frequent features receive smaller steps.

**Advantages:** Excellent for sparse data • Automatic learning-rate adaptation

**Limitations:** Learning rate continually shrinks and may become nearly zero

**Best for:** Sparse features, text, and recommendation systems



## 5 RMSprop

**How it works:** Divides the gradient by a moving average of recent squared gradients.

**Advantages:** Prevents Adagrad's rapid decay • Handles changing gradients

**Limitations:** Requires decay-rate tuning

**Best for:** Recurrent networks and non-stationary objectives



## 6 ADADELTA

**How it works:** Uses moving windows of squared gradients and squared updates.

**Advantages:** Reduces need for a manually chosen learning rate • Robust scaling

**Limitations:** Sometimes slower than Adam • Less commonly used today

**Best for:** Problems sensitive to learning-rate scale



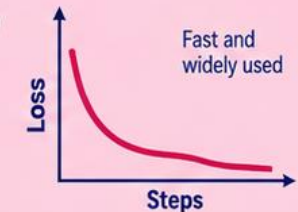
## 7 ADAM – ADAPTIVE MOMENT ESTIMATION

**How it works:** Combines Momentum (first moment) and RMSprop-like scaling (second moment).

**Advantages:** Fast • Adaptive • Works well with noisy or sparse gradients

**Limitations:** More memory • Can generalize worse than SGD in some tasks

**Best for:** Strong default choice for deep learning



### QUICK COMPARISON



$\eta$  = learning rate •  $\beta$  = momentum/decay factor •  $\nabla L$  = loss gradient  
Optimizers operate during TRAINING. Validation evaluates hyperparameters; it does not update weights.

Same idea. Brighter possibilities.

# 1 STOCHASTIC GRADIENT DESCENT (SGD)

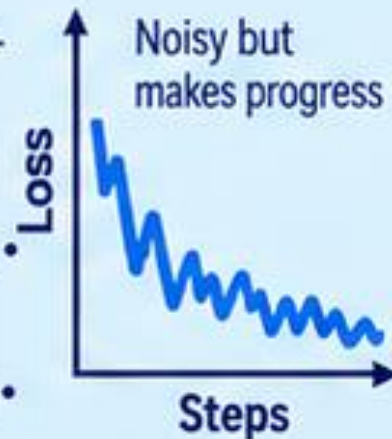
**How it works:** Updates weights using ONE randomly selected training example.

**Update:**  $w \leftarrow w - \eta \nabla L_i(w)$

**Advantages:** Fast updates • Low memory • Can escape shallow minima

**Limitations:** Noisy path • Loss fluctuates • May converge slowly

**Best for:** Very large or streaming datasets



# 2 MINI-BATCH GRADIENT DESCENT

**How it works:** Updates weights using a small batch, commonly 32–256 examples.

**Update:**  $w \leftarrow w - \eta (\text{average batch gradient})$

**Advantages:** Stable and efficient • GPU-friendly • Most widely used

**Limitations:** Batch size must be selected • Still has some noise

**Best for:** Most deep-learning training



### 3 MOMENTUM

**How it works:** Adds a fraction of the previous update to build speed.

$$v_t \leftarrow \beta v_{t-1} + \nabla L(w_t)$$

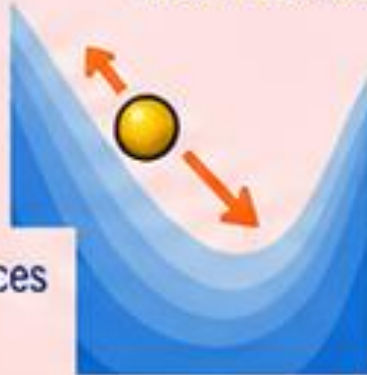
$$w_{t+1} \leftarrow w_t - \eta v_t$$

**Advantages:** Faster convergence • Reduces zig-zagging

**Limitations:** Extra  $\beta$  hyperparameter • May overshoot

**Best for:** Ravines and consistent gradient directions

Builds momentum down ravines



### 4 ADAGRAD

**How it works:** Gives each parameter its own learning rate; frequent features receive smaller steps.

**Advantages:** Excellent for sparse data • Automatic learning-rate adaptation

**Limitations:** Learning rate continually shrinks and may become nearly zero

**Best for:** Sparse features, text, and recommendation systems



## 5 RMSprop

**How it works:** Divides the gradient by a moving average of recent squared gradients.

**Advantages:** Prevents Adagrad's rapid decay • Handles changing gradients

**Limitations:** Requires decay-rate tuning

**Best for:** Recurrent networks and non-stationary objectives



## 6 ADADELTA

**How it works:** Uses moving windows of squared gradients and squared updates.

**Advantages:** Reduces need for a manually chosen learning rate • Robust scaling

**Limitations:** Sometimes slower than Adam • Less commonly used today

**Best for:** Problems sensitive to learning-rate scale



Adapts using moving windows

## 7 ADAM – ADAPTIVE MOMENT ESTIMATION

**How it works:** Combines Momentum (first moment) and RMSprop-like scaling (second moment).

**Advantages:** Fast • Adaptive • Works well with noisy or sparse gradients

**Limitations:** More memory • Can generalize worse than SGD in some tasks

**Best for:** Strong default choice for deep learning



# Thank You

# Influence of model weight & Need of Regularization

A weight tells the model how strongly a feature affects the prediction

**Predicted marks**

=

**5 × study hours**

**+ 0.35 × attendance**

**+ 1.8 × practice tests**

The numbers 5, 0.35 and 1.8 are weights. Training selects these values to reduce prediction error.

**Large weight - ???**  
**Small weight - ???**

# Influence of model weight & Need of Regularization

A weight tells the model how strongly a feature affects the prediction

**Predicted marks**

**=**

**5 × study hours**

**+ 0.35 × attendance**

**+ 1.8 × practice tests**

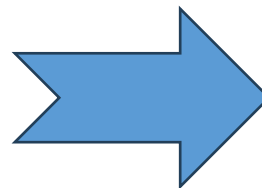
The numbers 5, 0.35 and 1.8 are weights. Training selects these values to reduce prediction error.

**Large weight - The feature has a strong influence**

**Small weight - The feature has a weak influence**

The model must balance two goals:

- reduce prediction error
- keep weights reasonably small



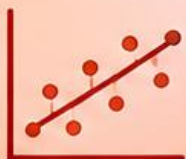
**Thus, Regularisation adds a cost for large weights keeping the weights in control**



# Linear Regression

Model a continuous target as a linear combination of features.

## Without regularisation



May overfit when features are many or highly correlated.

## With regularisation



Add a penalty to the loss function to constrain model coefficients and reduce overfitting.

### Ridge Regression — L2



L2

Shrinks coefficients; usually keeps all features.

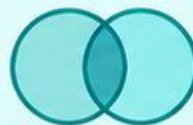
### Lasso Regression — L1



L1

Can reduce some coefficients to zero; performs feature selection.

### Elastic Net — L1 + L2



L1 + L2

Combines sparsity of Lasso with stability of Ridge.



Regularisation controls model complexity and helps improve generalisation.



REGULARISATION  
BUILDS SIMPLER,  
STRONGER MODELS

# L1, L2 & Elastic Net Regularisation

SIMPLER MODELS • BETTER GENERALISATION • MORE RELIABLE PREDICTIONS

Less  
Overfitting  
Brighter  
Insights

## L1 Regularisation – Lasso

SPARSE SOLUTIONS FOR A SIMPLER MODEL

$$\text{Loss} = \text{MSE} + \alpha \sum |w_j|$$



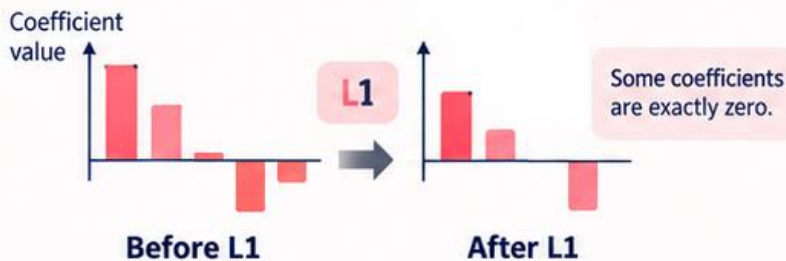
### Effect:

Shrinks coefficients; some can become exactly zero.



### Importance:

- Performs automatic feature selection
- Creates a simpler, interpretable model
- Useful when many features are irrelevant



## L2 Regularisation – Ridge

SHRINKS COEFFICIENTS FOR STABLE MODELS

$$\text{Loss} = \text{MSE} + \alpha \sum w_j^2$$



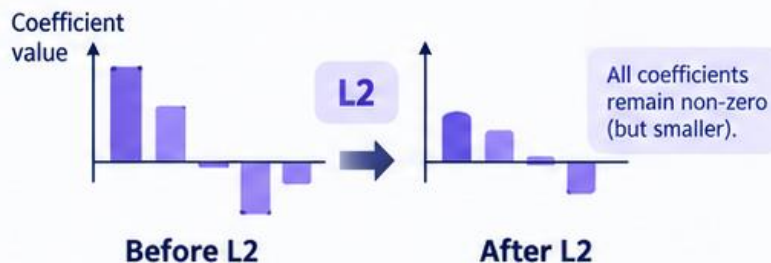
### Effect:

Shrinks all coefficients but usually keeps them non-zero.



### Importance:

- Controls large coefficient values
- Handles correlated features well
- Improves model stability and generalisation



## Elastic Net – L1 + L2

THE BEST OF BOTH WORLDS

$$\text{Loss} = \text{MSE} + \alpha \left[ r \sum |w_j| + (1-r) \sum w_j^2 \right]$$



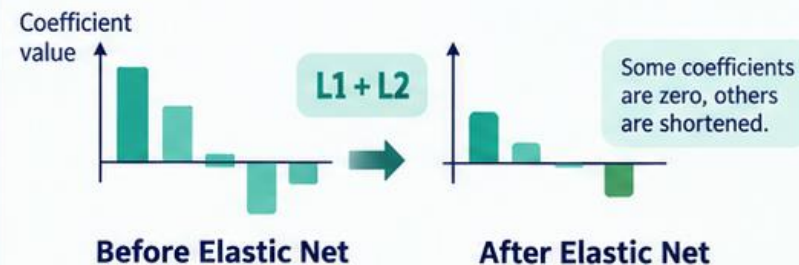
### Effect:

Combines feature selection from L1 with stability from L2.



### Importance:

- Useful for many correlated features
- Selects features while controlling coefficients
- Balances sparsity and stability



**Regularisation reduces overfitting by controlling model complexity.**

$\alpha$  controls penalty strength;  
 $r$  controls the L1–L2 balance.

# Worked Example: Regularisation of a Student Marks Model



Study  
Plan  
Succeed!

Given model:

Predicted marks =  $5 \times \text{study hours} + 0.35 \times \text{attendance} + 1.8 \times \text{practice tests}$

Weights:  $w_1 = 5$ ,  $w_2 = 0.35$ ,  $w_3 = 1.8$

Find the L1, L2, and Elastic Net penalties.



More  
Practice  
Brighter  
Results!

## 1. L1 Penalty

$$\begin{aligned} P_1 &= |w_1| + |w_2| + |w_3| \\ &= |5| + |0.35| + |1.8| \\ &= 5 + 0.35 + 1.8 \end{aligned}$$

$$P_1 = 7.15$$

Encourages some weights to become zero.

## 2. L2 Penalty

$$\begin{aligned} P_2 &= w_1^2 + w_2^2 + w_3^2 \\ &= 5^2 + 0.35^2 + 1.8^2 \\ &= 25 + 0.1225 + 3.24 \end{aligned}$$

$$P_2 = 28.3625$$

Shrinks all weights, especially large ones.

## 3. Elastic Net Penalty

Assume  $r = 0.5$

$$\begin{aligned} PEN &= rP_1 + (1 - r)P_2 \\ &= 0.5(7.15) + 0.5(28.3625) \\ &= 3.575 + 14.18125 \end{aligned}$$

$$PEN = 17.75625$$

Combines feature selection and coefficient stability.



If  $\alpha$  is the regularisation strength:

$$\text{L1 loss} = \text{MSE} + \alpha(7.15)$$

$$\text{L2 loss} = \text{MSE} + \alpha(28.3625)$$

$$\text{Elastic Net loss} = \text{MSE} + \alpha(17.75625)$$

Small  
Changes  
Big  
Progress!



Penalty values can be calculated from the weights. New regularised weights require retraining the model on data.

# Gradient Descent Without Regularisation

1. Start with old weight

$$W = 5$$

2. Calculate error gradient

$$\text{Error gradient} = 2$$

Only the prediction error is used.

3. Update the weight

Learning rate  $\eta = 0.1$

$$W_{\text{new}} = W_{\text{old}} - \eta(\text{Error gradient})$$

$$W_{\text{new}} = 5 - 0.1(2)$$

4. New weight

$$W_{\text{new}} = 4.8$$

$$5.0 \rightarrow 4.8$$

Without regularisation, the weight is updated only to reduce prediction error.

No penalty term

**Takeaway:** Gradient descent repeatedly moves the weight in the direction that reduces prediction error.

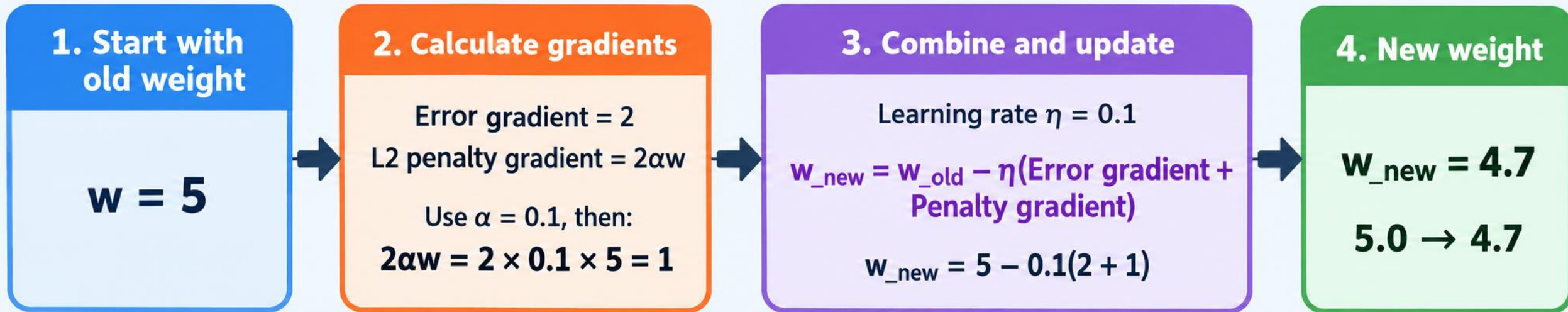
RECALL

## Gradient Descent Without Regularisation: Moving Down the Loss Curve



# Gradient Descent With Regularisation

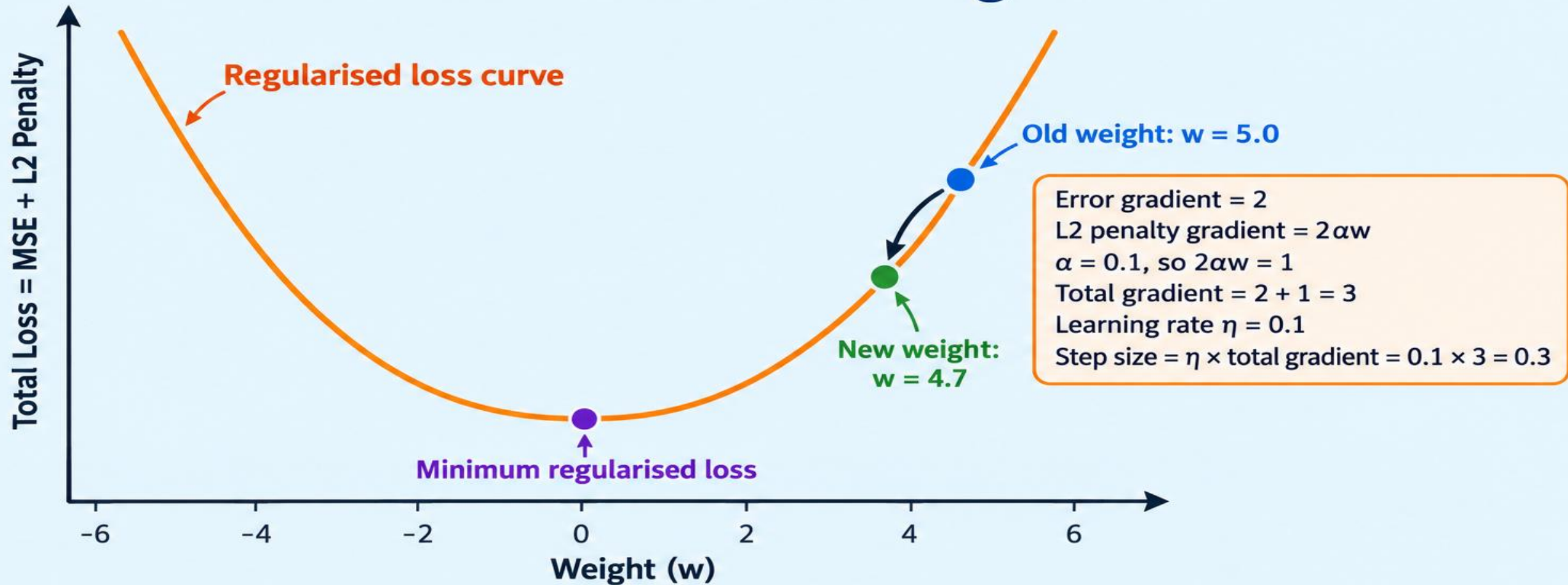
## How Gradient Descent Creates a New Weight



Prediction error moves the weight toward a better fit; regularisation also pulls it toward zero.

Illustrative one-step L2 update.

# Gradient Descent With L2 Regularisation



$$w_{\text{new}} = w_{\text{old}} - \eta(\text{Error gradient} + \text{Penalty gradient})$$
$$w_{\text{new}} = 5.0 - 0.1(2 + 1) = 4.7$$

L2 pulls the weight toward zero

With L2 regularisation, gradient descent reduces prediction error and discourages large weights.

# Hyperparameters of Gradient Descent

Small choices.  
Big learning.



## Learning Rate ( $\eta$ )

Controls the size of each weight update.

Too small: slow |  
Too large: may overshoot

Examples: 0.1, 0.01, 0.001



## Epochs

Maximum number of complete passes through the data.

Too few: underfitting |  
Too many: wasted training

Examples: 100, 500, 1000



## Batch Size

Number of samples used for one weight update.

1 = SGD | 32–256 = Mini-batch |  
All = Batch GD



## Learning-Rate Schedule

Changes the learning rate during training.

Types: constant, inverse scaling, adaptive



## Tolerance

Minimum improvement required to continue.

Smaller value allows more precise convergence.

Examples:  $10^{-3}$ ,  $10^{-4}$



## Patience

Number of epochs allowed without improvement.

Used for early stopping.

Examples: 5, 10, 20



## Regularisation Strength ( $\alpha$ )

Controls the penalty on large weights.

Too small: weak control |  
Too large: underfitting

Examples: 0.0001, 0.001, 0.01



## Momentum

Uses previous updates to accelerate learning.

Helps reduce oscillation.

Examples: 0.9, 0.99



## Weight update:

$$w_{\text{new}} = w_{\text{old}} - \eta \times \text{gradient}$$



## The most influential choices are

learning rate, epochs, batch size, and regularisation strength.

# The Role of alpha –

Alpha controls how strongly the penalty influences training

$$\text{Total loss} = \text{prediction error} + \alpha \times \text{regularisation penalty}$$

## Small $\alpha$

Weak penalty  
Weights remain larger  
Higher overfitting risk

## Suitable $\alpha$

Balanced penalty  
Useful patterns remain  
Better validation result

## Very large $\alpha$

Strong penalty  
Weights become tiny or zero  
Higher underfitting risk

**Alpha changes the strength of regularisation. It does not directly assign a weight value.**

# Ridge regression

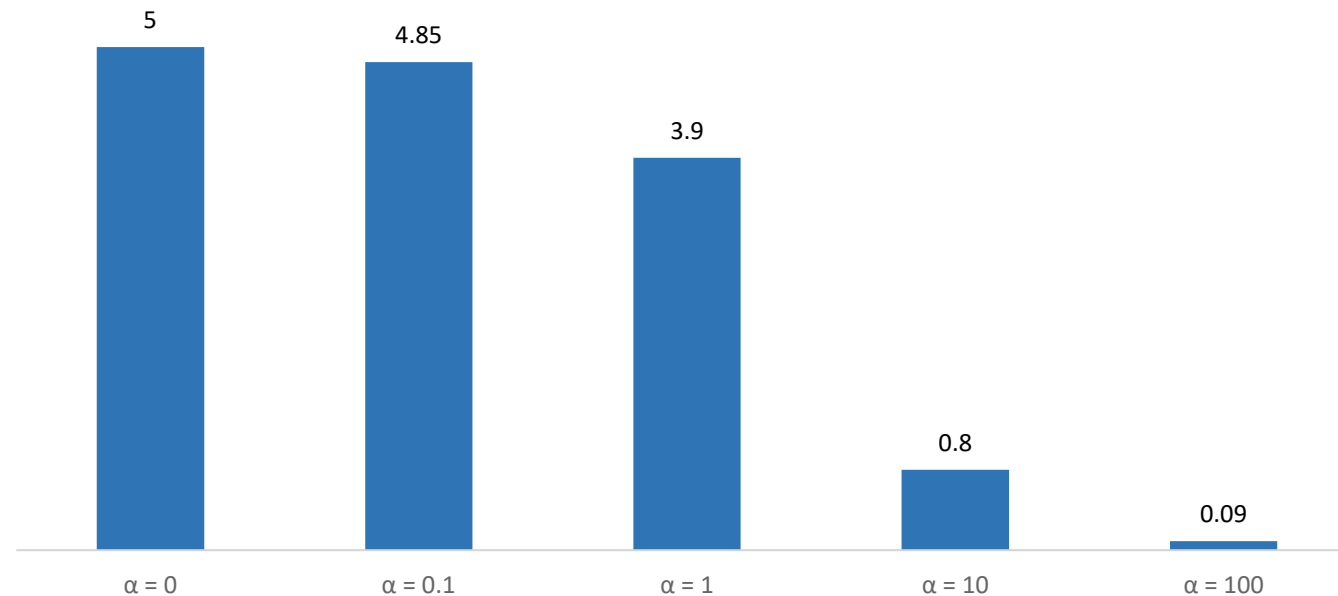
L2 regularisation shrinks every coefficient toward zero

$$\text{Penalty} = \alpha \times \Sigma(\text{weight}^2)$$

Ridge usually keeps all features

As alpha increases, coefficients become smaller. They normally approach zero without becoming exactly zero.

Useful when many features contribute to the prediction



# Lasso regression

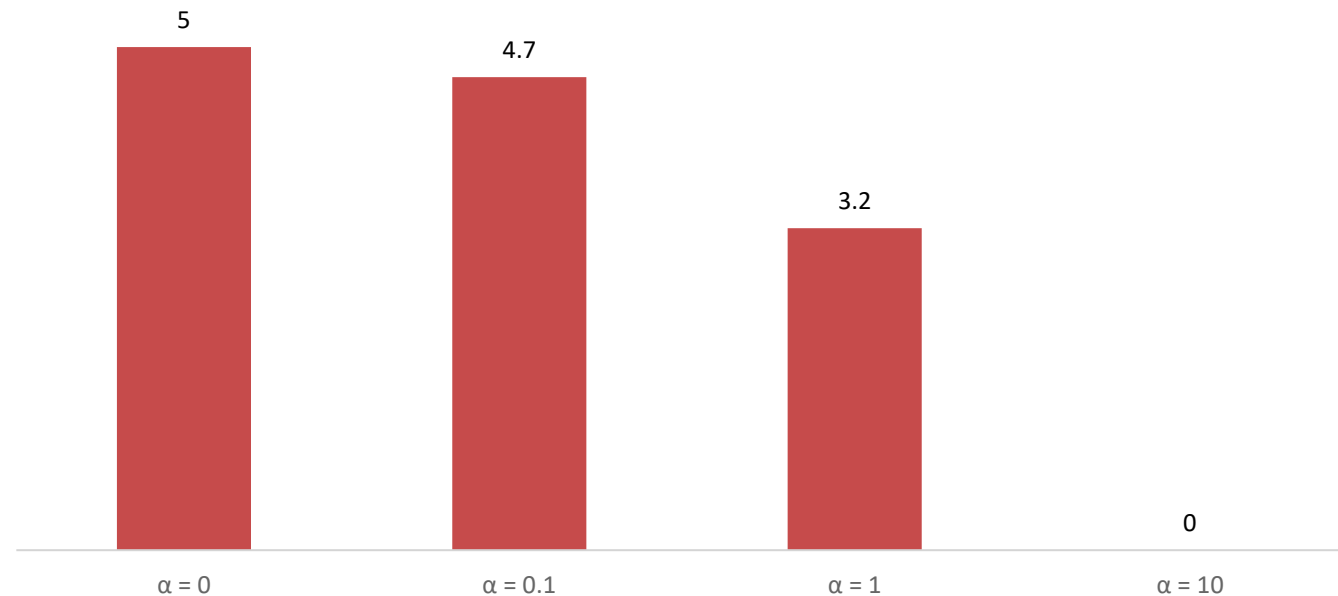
L1 regularisation can make weak coefficients exactly zero

$$\text{Penalty} = \alpha \times \sum |\text{weight}|$$

**Lasso can remove weak features**

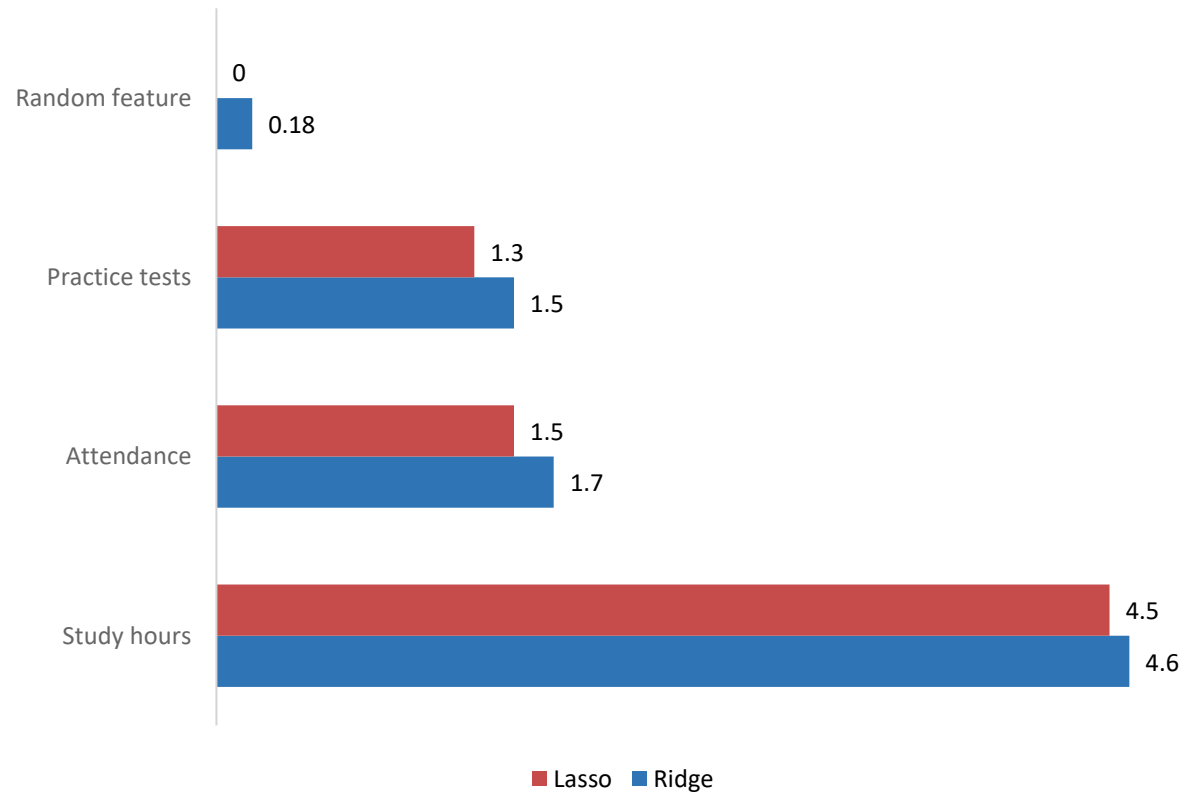
When a coefficient reaches zero, that feature no longer contributes to the prediction. This creates automatic feature selection.

**Useful when only some features matter**



# Ridge and Lasso compared

The same alpha can affect the two penalties differently



## Ridge

Shrinks all weights  
Usually retains every feature  
Works well when many features matter

## Lasso

Can create zero weights  
Selects a smaller feature set  
Works well when few features matter

# Alpha and the optimizer

The optimizer searches for weights that minimise the penalised loss

1

Start with candidate weights

2

Calculate prediction error and penalty

3

Update weights to reduce total loss

4

Stop when the solver converges

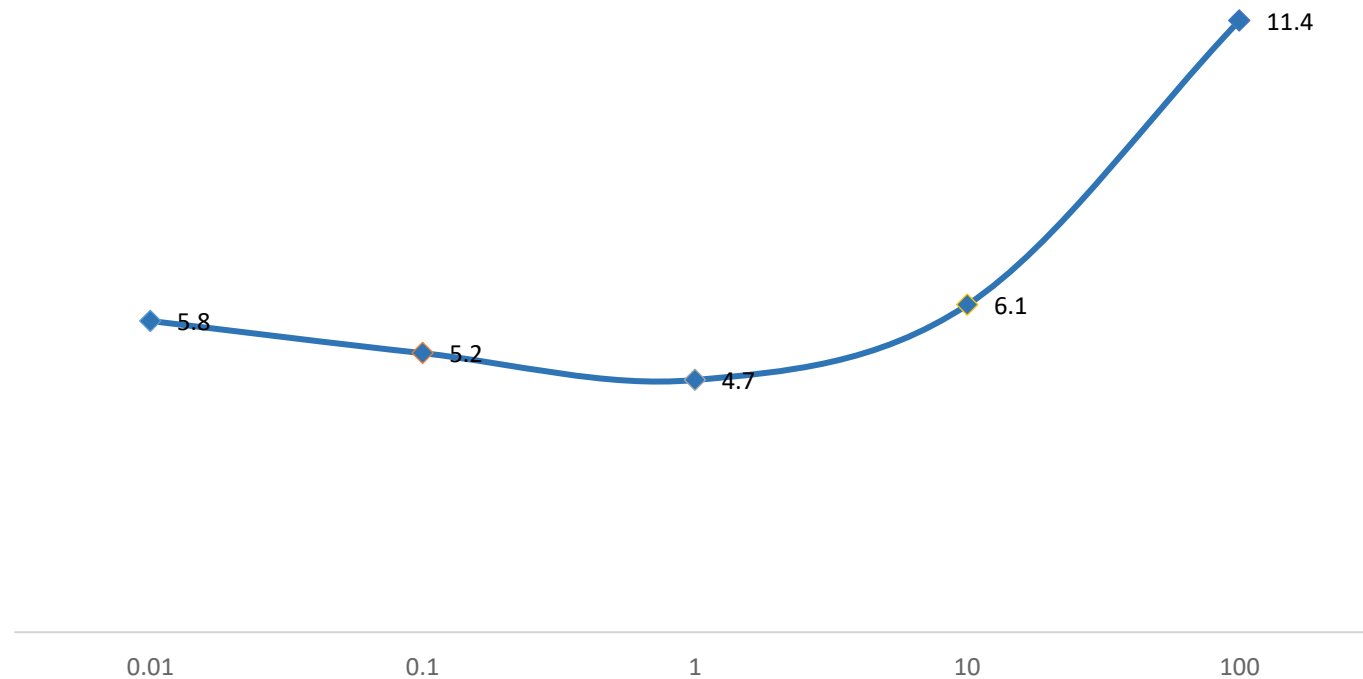
**Alpha changes the penalty**

Learning rate changes the step size for gradient based optimizers

Some Ridge and Lasso solvers do not expose a learning rate because they use other numerical methods.

# Validation results

Choose alpha from performance on unseen validation data



**Lowest RMSE**

The model makes the smallest typical validation error near  $\alpha = 1$  in this example.

Training error alone may favour too little regularisation. Validation data reveals overfitting and underfitting.

## Key points and check for understanding

### Alpha

Controls regularisation strength

### Ridge

Shrinks weights toward zero

### Lasso

Can make weights exactly zero

### Optimizer

Finds weights that minimise total loss

- 1 Why can Lasso remove a feature while Ridge normally keeps it?
- 2 What happens when alpha becomes too large?
- 3 How is alpha different from a learning rate?



REGULARISATION  
BUILDS SIMPLER,  
STRONGER MODELS

# L1, L2 & Elastic Net Regularisation

SIMPLER MODELS • BETTER GENERALISATION • MORE RELIABLE PREDICTIONS

Less  
Overfitting  
Brighter  
Insights

## L1 Regularisation – Lasso

SPARSE SOLUTIONS FOR A SIMPLER MODEL

$$\text{Loss} = \text{MSE} + \alpha \sum |w_j|$$



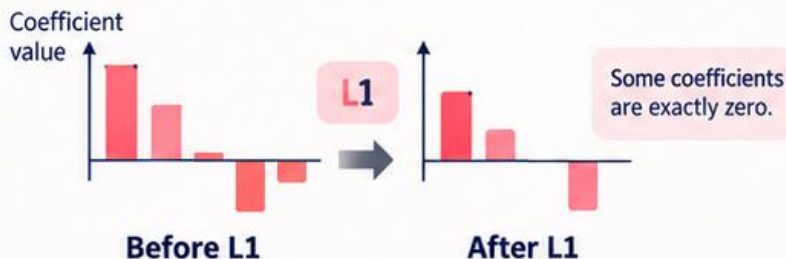
### Effect:

Shrinks coefficients; some can become exactly zero.



### Importance:

- Performs automatic feature selection
- Creates a simpler, interpretable model
- Useful when many features are irrelevant



## L2 Regularisation – Ridge

SHRINKS COEFFICIENTS FOR STABLE MODELS

$$\text{Loss} = \text{MSE} + \alpha \sum w_j^2$$



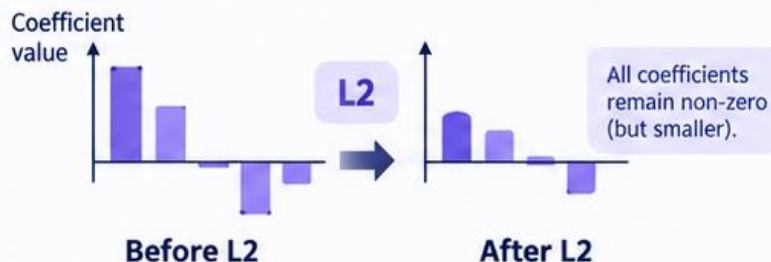
### Effect:

Shrinks all coefficients but usually keeps them non-zero.



### Importance:

- Controls large coefficient values
- Handles correlated features well
- Improves model stability and generalisation



## Elastic Net – L1 + L2

THE BEST OF BOTH WORLDS

$$\text{Loss} = \text{MSE} + \alpha \left[ r \sum |w_j| + (1-r) \sum w_j^2 \right]$$



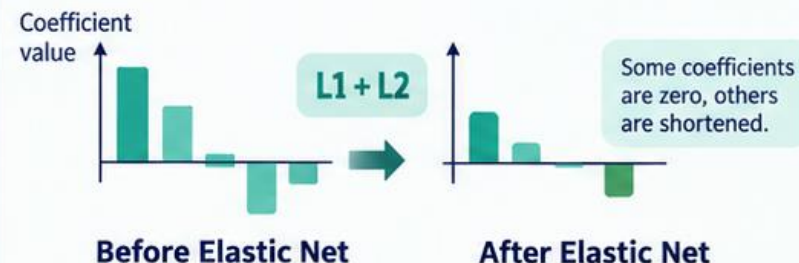
### Effect:

Combines feature selection from L1 with stability from L2.



### Importance:

- Useful for many correlated features
- Selects features while controlling coefficients
- Balances sparsity and stability



**Regularisation reduces overfitting by controlling model complexity.**

$\alpha$  controls penalty strength;  
 $r$  controls the L1–L2 balance.

| Type of regression         | How validation is useful                                                                       |
|----------------------------|------------------------------------------------------------------------------------------------|
| Simple Linear Regression   | Checks whether the relationship between one input and the target works on unseen data.         |
| Multiple Linear Regression | Helps select useful predictors and identify models with unnecessary features.                  |
| Polynomial Regression      | Selects the appropriate polynomial degree and detects overfitting.                             |
| Ridge Regression           | Selects the best regularization strength, $\lambda$ or alpha.                                  |
| Lasso Regression           | Tunes regularization strength and controls how many features are removed.                      |
| Elastic Net Regression     | Tunes both the regularization strength and the balance between Ridge and Lasso penalties.      |
| Robust Linear Regression   | Helps determine whether the model generalizes well when the dataset contains outliers.         |
| Bayesian Linear Regression | Evaluates the model's predictive performance and the reliability of its uncertainty estimates. |